

Project 4: Unsupervised Image Clustering and Visualization

(K-Means Clustering, PCA and t-SNE on the MNIST Handwritten Digits)

1. Executive Summary

This project asks whether unsupervised learning can discover groups in 70,000 handwritten digit images (MNIST) without being told which digit each image shows. Following the updated instructions, K-Means was trained on a 10,000-image sample (elbow comparison of $K = 5, 10$ and 15), interpreted with centroids and a crosstab against the true labels, visualized with PCA, and used to assign new images. An extended analysis repeated the work on all 60,000 training images, validated it on unseen data, and compared PCA with t-SNE. Short answer: yes, K-Means finds meaningful groups, but not a clean one-cluster-per-digit split.

Question	Answer
Best number of clusters?	No sharp elbow; k = 10 chosen (gains flatten near $k = 8-10$; matches the 10 digits)
How well do clusters match digits?	~ 59% accuracy (majority mapping); ARI 0.367 , NMI 0.498 on the test set
Does the model generalize?	Yes. Mean distance 6.161 (train) vs 6.163 (test); 95.2% of test images fit their cluster
Hardest digits?	5 and 9 never get their own cluster (0% accuracy); 4/7/9 and 3/5/8 get mixed
Easiest digits?	1 (99.5%), 0 (88.9%), 6 (83.0%)
Best 2D view?	t-SNE (trustworthiness 0.982 vs 0.739 for PCA)
Biggest improvement found?	K-Means on the t-SNE map: accuracy 58.0% → 81.9% , ARI 0.364 → 0.713

Main lesson: the digits are clearly separable, but not by straight-line pixel distance. How the data is represented matters more than the clustering algorithm itself.

2. Objective and Data

2.1 Objective

Discover groups in the MNIST images with K-Means, judge how meaningful those groups are, show how the trained model assigns new images, and use dimensionality reduction to visualize and assess the clustering. Labels were used **only for evaluation**, never for training.

2.2 Datasets

	MNIST (main)	load_digits (external test)
Images	70,000 (60,000 train / 10,000 test)	1,797

	MNIST (main)	load_digits (external test)
Resolution	28 x 28	8 x 8 (upscaled to 28 x 28)
Features	784	64
Raw pixel range	0-255	0-16
Source	fetch_openml("mnist_784")	sklearn.datasets.load_digits

Tools: Python 3, NumPy, pandas, scikit-learn (KMeans, PCA, TSNE, metrics), SciPy, matplotlib, seaborn, Plotly, Pillow and joblib.

3. Methodology

3.1 Data Preparation

- Loaded the data with `fetch_openml` and kept the standard 60,000 / 10,000 train-test split.
- Checked quality with assert statements: no missing values, shape 70,000 x 784, pixels within 0-255.
- Normalized pixels to 0-1 (divide by 255) so all features share one scale. This matters because K-Means uses Euclidean distance.
- Flattened each 28 x 28 image into a 784-value vector and verified the round trip.

3.2 Exploratory Data Analysis

Class balance, sample images, mean image per digit, pixel intensity distribution, pixel variance map, ink per digit, and PCA explained variance.

3.3 Clustering

- **Required workflow:** drew a 10,000-image sample from all 70,000 images (seed 42), compared K = 5, 10 and 15 with the elbow method, trained the final K = 10 model on the sample, and analyzed its centroids, member images, crosstab with the labels, PCA scatter and predictions for 10 unseen images (Section 6).
- **Extended analysis:** the steps below.
- Searched k = 2 to 15 on a 10,000-image sample using the elbow method (inertia) and the silhouette score (on a 3,000-image subset).
- Trained the final K-Means model (k = 10, n_init = 10, random_state = 42) on all 60,000 training images.
- Interpreted clusters with centroid images, a cluster-vs-digit table, purity, and sample images per cluster.

3.4 Evaluation and Prediction

- Built reusable functions: `preprocess_images()` for any input shape, `predict_cluster()` returning cluster, distance, runner-up and confidence margin, and `to_mnist_format()` for real image files.
- Defined a per-cluster "good fit" threshold at the 95th percentile of training distances.

- Validated on the unseen test set, the external load_digits set, noisy images ($\sigma = 0.1$ to 0.5) and a custom image file.
- Scored with majority-mapping accuracy, Adjusted Rand Index (ARI), Normalized Mutual Information (NMI) and a confusion matrix.

3.5 Dimensionality Reduction and Visualization

- PCA to 2D for plotting and to 50D as input for t-SNE; t-SNE to 2D (perplexity 30) on a 5,000-image sample.
- Compared views with silhouette score and trustworthiness; mapped mismatches and low-confidence points; analyzed centroid distances and a dendrogram.
- Tested t-SNE perplexity 5, 30 and 50, and re-ran K-Means inside each reduced space.

4. Key Decisions

Decision	Reason
Normalize pixels to 0-1	Puts every feature on one scale for distance-based K-Means
Keep the official 60k / 10k split	Standard benchmark; test images stay truly unseen
Sample 10k (elbow), 3k (silhouette), 5k (t-SNE)	Large time savings while staying representative
k = 10	No sharp elbow; gains flatten near 10; matches the 10 digit classes
Majority mapping for scoring	Simple, explainable way to compare clusters with labels
Per-cluster 95th percentile fit threshold	Clusters differ in spread (7.07 to 8.35), so one global cut-off would be unfair
PCA to 50D before t-SNE	Standard practice; removes noise and speeds up t-SNE
random_state = 42 throughout	Reproducible results
Save model, mapping and thresholds	Predictions work later without retraining

5. Analysis and Results

5.1 Exploratory Data Analysis

- **Balanced classes:** each digit is 8.9% to 11.4% of the data, so no resampling was needed.
- **Sparse images:** about 81% of pixels are background, and 67 of 784 pixels are zero in every training image.
- **Redundant features:** PCA keeps 80% of the variance with 44 components, 95% with 154 and 99% with 331.
- **Ink varies by digit:** 1 uses the least ink (mean 59.6); 0 uses the most (135.9).

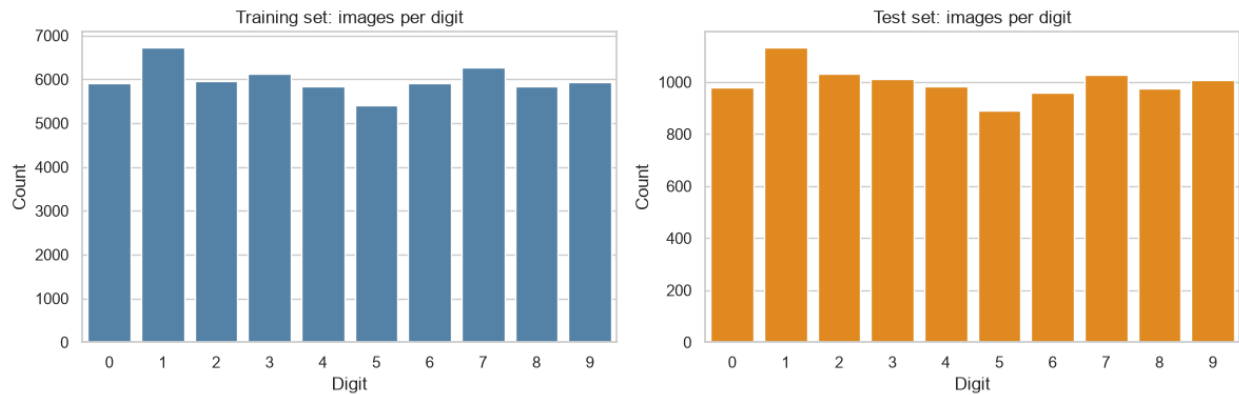


Figure 1. Images per digit in the training and test sets.

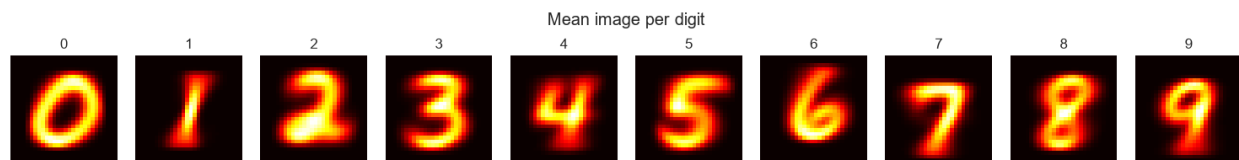


Figure 2. Mean image per digit.

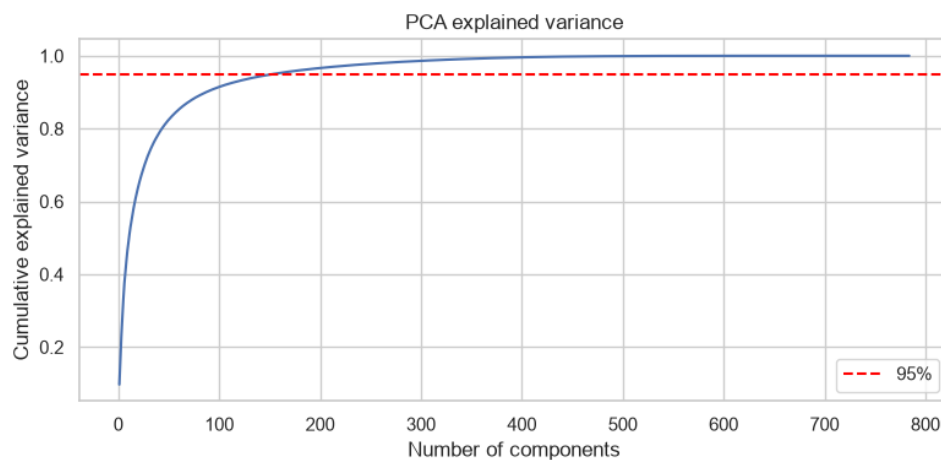


Figure 3. Cumulative PCA explained variance.

5.2 Choosing the Number of Clusters

The elbow curve bends gradually with **no sharp elbow**. The inertia drop per extra cluster falls from 4.9% at $k = 3$ to about 1.4% from $k = 10$ onward. Silhouette scores are very low for every k (0.05 to 0.08), which is typical for raw pixel data: in 784 dimensions all images are far apart and digits blend into each other. The highest silhouette is at $k = 2$, which is too coarse to be useful. **$k = 10$** was chosen because the curve flattens there and it matches the known number of digits.

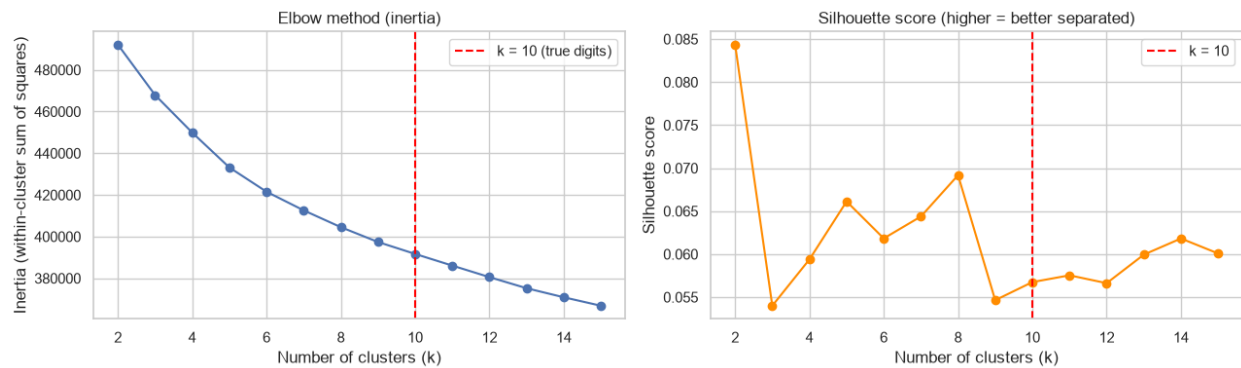


Figure 4. Elbow method (left) and silhouette score (right) for k = 2 to 15.

5.3 Cluster Interpretation

The final model trained in 45.5 seconds (inertia 2,352,828). Cluster sizes range from 3,098 to 8,929 images even though the digits are balanced, which is the first sign that clusters do not map one-to-one to digits.

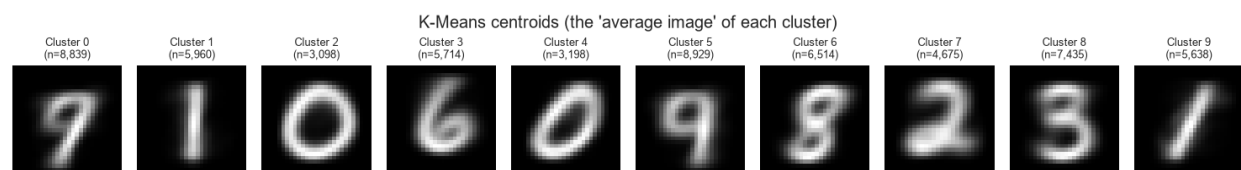


Figure 5. K-Means centroids: the average image of each cluster.

Cluster	Size	Majority digit	Purity	Second digit	Avg ink
C2	3,098	0	90.7%	6 (3.2%)	139.4
C4	3,198	0	78.1%	5 (8.6%)	139.5
C1	5,960	1	62.4%	3 (7.1%)	61.0
C9	5,638	1	52.8%	5 (14.3%)	71.6
C7	4,675	2	89.7%	3 (4.6%)	124.3
C8	7,435	3	52.8%	5 (23.8%)	114.7
C5	8,929	4	35.7%	9 (32.5%)	95.1
C3	5,714	6	86.0%	2 (3.7%)	109.9
C0	8,839	7	42.7%	9 (27.8%)	94.1
C6	6,514	8	53.1%	5 (22.7%)	116.5

- **Digits split by writing style:** C2 is a round, upright 0 and C4 a slanted 0; C1 is an upright 1 and C9 a slanted 1.
- **Look-alike digits share clusters:** C5 are almost equally 4 and 9; C0 mixes 7 and 9; C8 and C6 each absorb many 5s.
- **5 and 9 have no cluster of their own.** Because 0 and 1 used two clusters each, only eight clusters were left for the other eight digits.

- **Ink drives part of the grouping:** the 1-clusters have the lowest average ink, the 0-clusters the highest.

Cluster vs true digit (training set)

cluster \ true_digit	0	1	2	3	4	5	6	7	8	9
0	14	9	69	48	1937	349	1	3773	179	2460
1	3	3718	362	425	159	162	269	326	297	239
2	2811	0	11	15	10	62	99	15	38	37
3	182	8	211	55	165	121	4913	4	47	8
4	2498	0	99	127	12	275	122	14	32	19
5	38	6	173	175	3187	378	82	1792	196	2902
6	177	10	158	1036	20	1481	90	11	3462	69
7	17	9	4194	214	37	14	84	39	54	13
8	162	5	328	3927	1	1770	29	5	1124	84
9	21	2977	353	109	314	809	229	286	422	118

Figure 6. Cluster vs true digit (training set).

5.4 Evaluation and Validation

Each cluster was labeled with its majority digit to score the model like a classifier. Train accuracy was **59.1%** and test accuracy **59.5%**, with test ARI **0.367** and NMI **0.498**: far above random (10%), but well below a supervised classifier.

Dataset	Images	Accuracy	Good fit	Mean distance	Mean margin
Train (reference)	60,000	59.1%	95.0%	6.161	0.145
Test (unseen MNIST)	10,000	59.4%	95.2%	6.163	0.144
load_digits (external)	1,797	33.6%	100.0%	5.028	0.121

Generalization: test images fit the clusters just like training images. Per-cluster mean distances change by only $\pm 1.6\%$, and cluster shares match closely. The $\sim 59\%$ accuracy is a limit of K-Means on pixels, not overfitting.

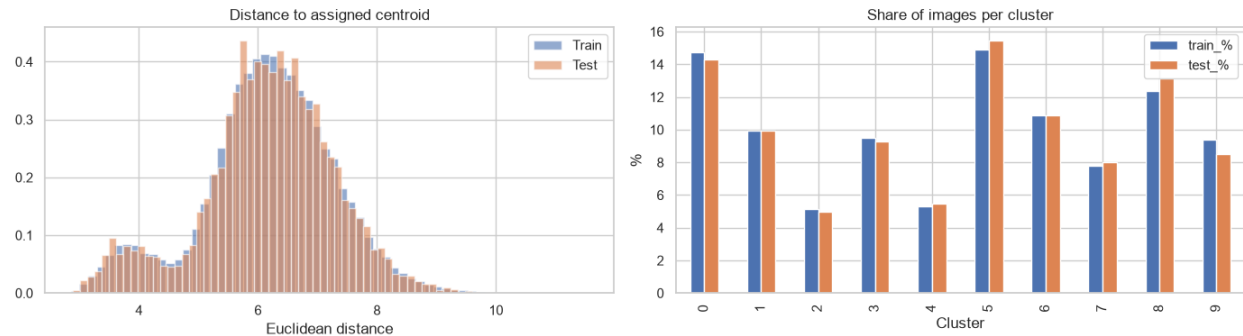


Figure 7. Distance to assigned centroid (left) and share of images per cluster (right), train vs test.

Per-digit accuracy (test): 1 (99.5%), 0 (88.9%), 6 (83.0%), 3 (68.9%), 2 (68.5%), 8 (60.2%), 7 (58.6%), 4 (56.9%), 5 (0%), 9 (0%). Digits 5 and 9 are structurally impossible to predict because no cluster has them as its majority.

Test set: true digit vs cluster prediction

True digit \ Predicted digit (via cluster)	0	1	2	3	4	5	6	7	8	9
0	871	5	2	48	4	0	30	3	17	0
1	0	1129	1	2	0	0	2	0	1	0
2	23	144	707	64	30	0	25	11	28	0
3	21	80	40	696	15	0	7	7	144	0
4	2	66	5	0	559	0	36	310	4	0
5	50	130	4	287	55	0	20	70	276	0
6	44	58	18	3	21	0	795	1	18	0
7	2	117	13	0	292	0	1	602	1	0
8	18	80	7	206	31	0	10	36	586	0
9	14	40	3	7	541	0	4	387	13	0

Figure 8. Test set: true digit vs cluster prediction.

External data: accuracy on load_digits drops to 33.6%, yet 100% of images count as a good fit. Upscaled 8 x 8 images are smooth and blurry, so they sit close to the (also blurry) centroids even when the cluster is wrong. The good-fit flag measures distance, not correctness, and cannot detect this kind of style shift.

Noise robustness:

Noise σ	Same cluster as clean	Accuracy	Good fit
0.0	100.0%	59.4%	95.2%
0.1	97.2%	60.0%	96.9%
0.2	94.2%	60.8%	93.8%

Noise σ	Same cluster as clean	Accuracy	Good fit
0.3	91.1%	61.1%	67.4%
0.5	83.9%	60.2%	0.0%

Cluster assignments stay stable under noise and accuracy stays near 60%. Unlike the external case, the fit check does catch heavily noisy images (0% good fit at $\sigma = 0.5$).

Custom image: a 7 saved as dark ink on white paper at 200 x 200 px was converted with `to_mnist_format()` and correctly assigned to cluster 0 (digit 7) with a margin of 0.33, well above the average of about 0.14.

5.5 Dimensionality Reduction and Visualization

Two PCA components keep only **16.8%** of the variance (PC1 9.7%, PC2 7.1%), while 50 components keep 82.5%. In the PCA view the digits overlap in one cloud; in the t-SNE view they form clear, separate islands.

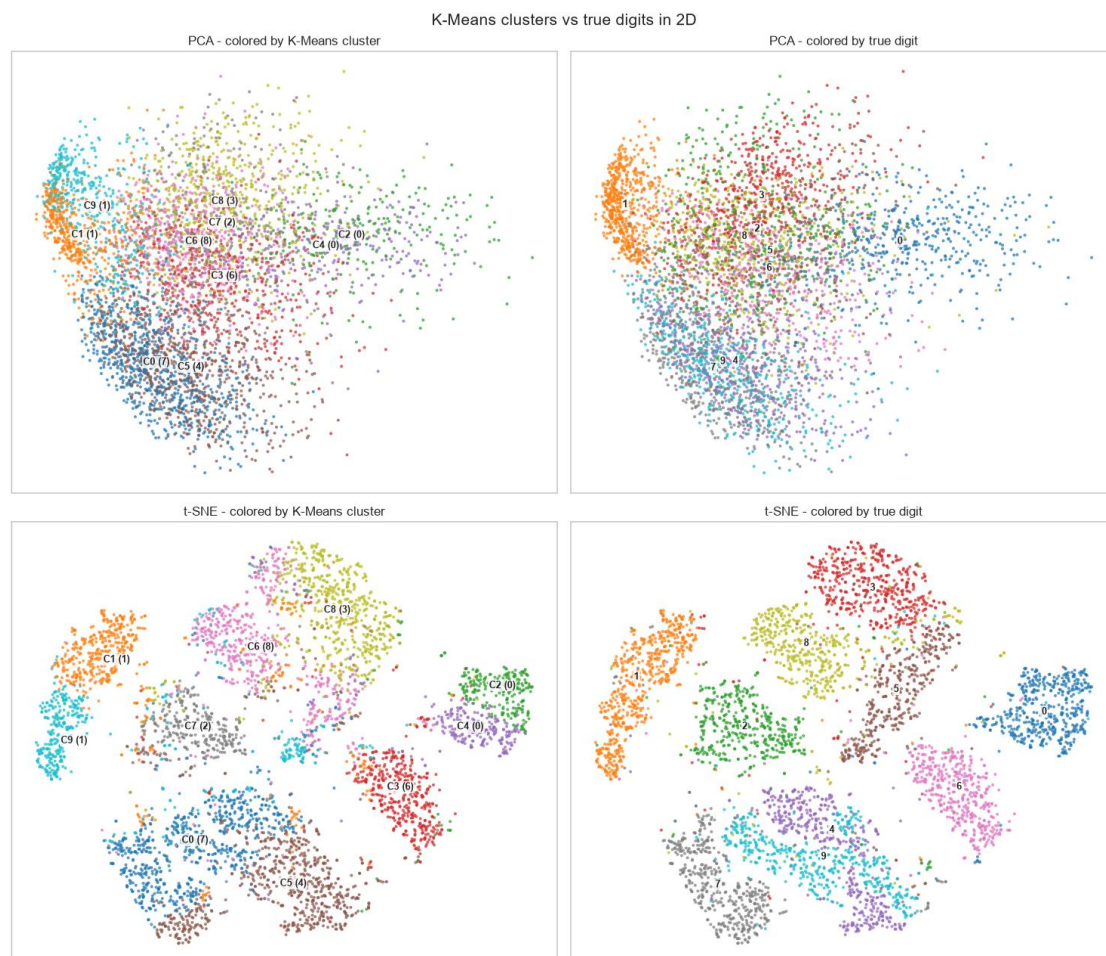


Figure 9. PCA (top) and t-SNE (bottom), colored by K-Means cluster (left) and true digit (right).

View	Silhouette (clusters)	Silhouette (true digits)	Trustworthiness
Original (784D)	0.059	0.046	1.000
PCA (2D)	-0.019	0.036	0.739
t-SNE (2D)	0.185	0.316	0.982

- t-SNE keeps 98% of real neighborhoods; PCA places many unrelated images next to each other.
- In t-SNE space the **true digits separate better (0.316)** than the **K-Means clusters (0.185)**: the structure is in the data, but K-Means draws the boundaries in the wrong places.

Where K-Means goes wrong: 41% of the 5,000 sampled points do not match their cluster's majority digit. The largest mix-ups are $9 \rightarrow 4$ (248), $9 \rightarrow 7$ (226), $4 \rightarrow 7$ (187), $7 \rightarrow 4$ (148), $5 \rightarrow 3$ (141), $5 \rightarrow 8$ (124) and $3 \rightarrow 8$ (92), forming two families of similar shapes: **{4, 7, 9}** and **{3, 5, 8}**.

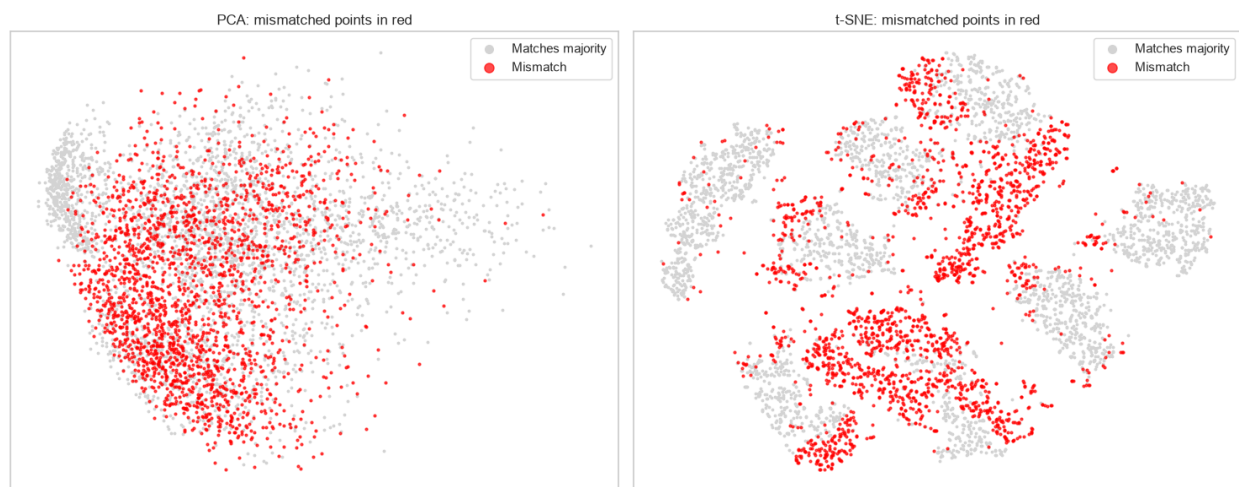


Figure 10. Points whose cluster majority does not match the true digit (red).

Per-cluster separation: C7 (2), C3 (6) and C2 (0) are the best separated. C4 (0) and C9 (1) have negative scores because upright and slanted versions of the same digit share one t-SNE island.

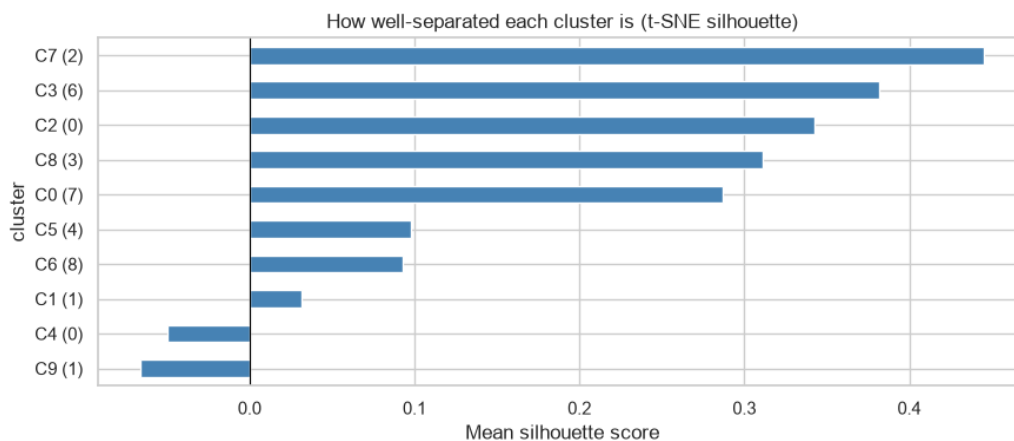


Figure 11. Mean t-SNE silhouette per cluster.

Relationships between clusters: the closest centroid pairs are C0 (7) ↔ C5 (4) at 3.98, C1 (1) ↔ C9 (1) at 4.02, C8 (3) ↔ C6 (8) at 4.24 and C7 (2) ↔ C3 (6) at 4.46. Among the least confident 10% of points, the most common competing pairs are C0 ↔ C5 (64 points) and C6 ↔ C8 (55 points), and these points sit on the borders between t-SNE islands.

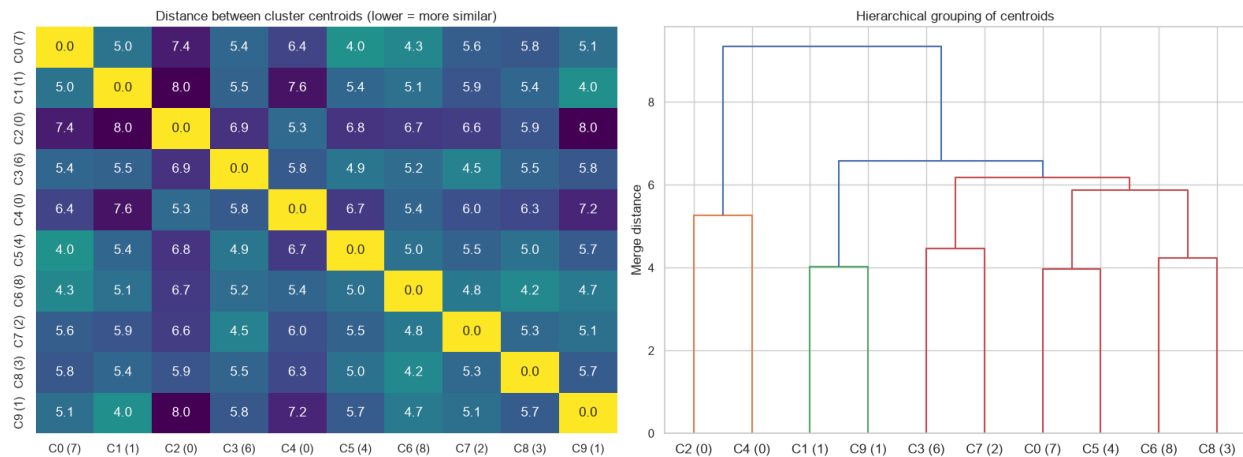


Figure 12. Distance between centroids (left) and hierarchical grouping of centroids (right).

Perplexity check: perplexity 5 breaks the islands into fragments, while 30 and 50 give clean, similar layouts, so the conclusions do not depend on one setting.

Clustering in reduced space:

Space	Accuracy	ARI	NMI
Pixels (784D)	58.0%	0.364	0.494
PCA (50D)	55.7%	0.341	0.478
PCA (2D)	41.5%	0.247	0.367
t-SNE (2D)	81.9%	0.713	0.767

PCA does not help, because it keeps the same linear structure K-Means already sees. t-SNE unfolds the curved digit shapes into compact islands, which nearly doubles ARI. Because t-SNE cannot transform new images, this is an analysis result rather than a deployable model.

6. Answers to the Project Questions (Parts 1–7)

This section answers every question in the updated Project 4 instructions. Parts 2 to 6 follow the required workflow: K-Means was trained on a **10,000-image sample drawn from all 70,000 images** (random seed 42), and the digit labels were **only used after clustering** to interpret the results. The notebook has a matching Questions and Answers cell after each part.

6.1 Part 1: Explore the Images

1. How many images are in the dataset?

70,000 images (shape 70,000 x 784), usually split into 60,000 training and 10,000 test images.

2. Why does each image have 784 features?

Each image is a 28 x 28 pixel grid and $28 \times 28 = 784$. The grid is flattened into one row of 784 numbers, so each pixel position becomes one feature.

3. What do the pixel values represent?

The brightness of each pixel: 0 = black background, 255 = brightest white ink. About 81% of all pixels are 0, so most of each image is empty background.

4. Why do images of the same digit look different?

Each digit was written by a different person, with a different slant, size, stroke thickness, position and style (for example a 7 with or without a crossbar, or a 4 with an open or closed top).

5. Why do we normalize the pixel values?

Dividing by 255 puts every pixel on the same 0–1 scale without changing the image. K-Means uses Euclidean distance, so one consistent, small scale makes distances comparable and the calculations more stable.

6.2 Part 2: Discover Groups Using K-Means

The elbow method compared $K = 5, 10$ and 15 on the 10,000-image sample ($n_init = 10$, $random_state = 42$).

K	Inertia	Drop from previous K	Drop per extra cluster
5	432,527	–	–
10	390,343	42,184 (–9.8%)	≈ 8,437
15	365,526	24,817 (–6.4%)	≈ 4,963

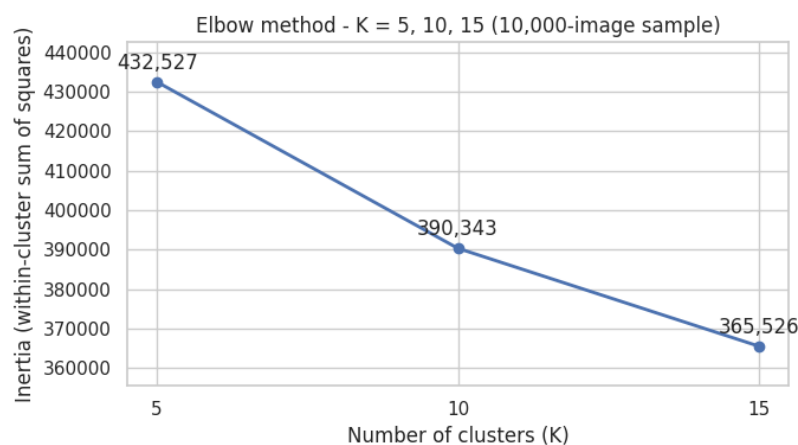


Figure 13. Elbow plot for $K = 5, 10$ and 15 (10,000-image sample).

1. Which value of K did you choose?

K = 10.

2. Why did you choose it?

The gain per extra cluster falls by about 40% after $K = 10$ (from about 8,400 to about 5,000 units of inertia), so the curve bends at 10. The longer $k = 2-15$ search also shows gains leveling off near 10, with no sharp elbow. $K = 10$ also matches the 10 digits, which makes the clusters easy to interpret. A larger K would mostly split digits further by writing style.

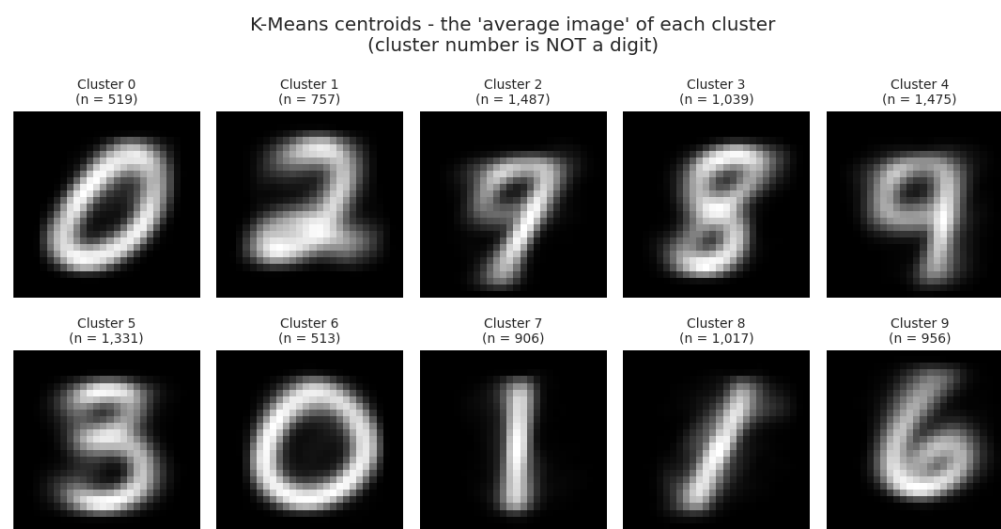
3. How many images are in each cluster?

Cluster	0	1	2	3	4	5	6	7	8	9
Images	519	757	1,487	1,039	1,475	1,331	513	906	1,017	956

4. Are the clusters approximately the same size?

No. They range from 513 to 1,487 images (2.9 times). Because the digits in the sample are balanced (906 to 1,152 each), equal clusters would hold about 1,000 images each. The two smallest clusters are both zeros, and the two largest mix 4, 7 and 9. This is the first sign that clusters do not match digits one-to-one.

6.3 Part 3: Understand the Clusters



Sharp centroids belong to consistent clusters: C1 (a 2), C6 (a round 0), C0 (a slanted 0), C7 (an upright 1), C8 (a slanted 1) and C9 (a 6). Blurry, blended centroids reveal mixed clusters: C2 looks like a 7 with a faint 9 loop, C4 like a 9 with 4 features, and C3 like an 8 blended with 3 and 5.

1. Do images within the same cluster look similar?

Mostly, at the level of overall shape. C1, C6, C7 and C9 are very consistent. C2, C3, C4 and C8 group images with a similar outline or slant but different digits (C4 holds 4s, 7s and 9s; C8 holds slanted 1s with thin, slanted 5s and 8s).

2. Does a cluster appear to contain mostly one digit?

Yes for five clusters: C6 and C0 (0s), C1 (2s), C9 (6s) and C7 (1s).

3. Are some digits spread across multiple clusters?

Yes. 0 is split into round (C6) and slanted (C0) versions, and 1 into upright (C7) and slanted (C8). 5 is spread across C5, C3 and C8, and 4, 7 and 9 are shared between C2 and C4.

4. Which digits appear difficult for K-Means to separate?

4, 7 and 9 (similar vertical strokes) and **3, 5 and 8** (similar right-side curves). 5 is the hardest.

6.4 Part 4: Compare Clusters with the Actual Digit Labels

K-Means cluster vs actual digit (10,000-image sample)

K-Means cluster	0	1	2	3	4	5	6	7	8	9
0	427	0	11	17	0	39	14	1	7	3
1	5	1	666	42	8	4	10	9	12	0
2	4	1	8	7	290	57	1	654	23	442
3	22	2	20	146	4	266	9	1	560	9
4	3	4	26	33	500	63	14	295	34	503
5	31	5	71	700	0	310	4	0	197	13
6	458	0	5	2	2	5	26	2	6	7
7	0	642	45	55	16	18	34	34	32	30
8	2	497	70	24	49	155	48	57	87	28
9	31	0	45	8	37	20	801	2	11	1
Actual digit	0	1	2	3	4	5	6	7	8	9

Figure 15. `pd.crosstab(clusters, y_sample): K-Means cluster vs actual digit.`

Cluster	Size	Most common digit	Purity	Top 3 digits
C6	513	0	89.3%	0 (89%), 6 (5%), 9 (1%)
C1	757	2	88.0%	2 (88%), 3 (6%), 8 (2%)
C9	956	6	83.8%	6 (84%), 2 (5%), 4 (4%)
C0	519	0	82.3%	0 (82%), 5 (8%), 3 (3%)
C7	906	1	70.9%	1 (71%), 3 (6%), 2 (5%)
C3	1,039	8	53.9%	8 (54%), 5 (26%), 3 (14%)
C5	1,331	3	52.6%	3 (53%), 5 (23%), 8 (15%)
C8	1,017	1	48.9%	1 (49%), 5 (15%), 8 (9%)
C2	1,487	7	44.0%	7 (44%), 9 (30%), 4 (20%)
C4	1,475	9	34.1%	9 (34%), 4 (34%), 7 (20%)

1. Does each cluster mainly contain one digit?

Only half do. Five clusters are 71%–89% one digit; the other five are mixed (34%–54%). C4 is almost a tie between 9 (503 images) and 4 (500).

2. Which digits are grouped together?

{4, 7, 9}: C2 holds 654 sevens, 442 nines and 290 fours; C4 holds 503 nines, 500 fours and 295 sevens. **{3, 5, 8}**: C5 holds 700 threes, 310 fives and 197 eights; C3 holds 560 eights, 266 fives and 146 threes.

3. Which digits appear in multiple clusters?

0 and 1 (split by style), 5 (at most 33% in any cluster), 4, 7, 9, 3 and 8. Only 2 (69%) and 6 (83%) stay mostly in one cluster. **4 and 5 are not the majority of any cluster.**

4. Why might some digits be difficult to separate?

K-Means compares images pixel by pixel with straight-line distance. Look-alike digits share many ink pixels, while a slanted and an upright version of the same digit share fewer. K-Means has no idea of shape and prefers round, similar-sized groups. Because 0 and 1 used two clusters each, only six clusters were left for the other eight digits.

6.5 Part 5: Visualize the Clusters with PCA

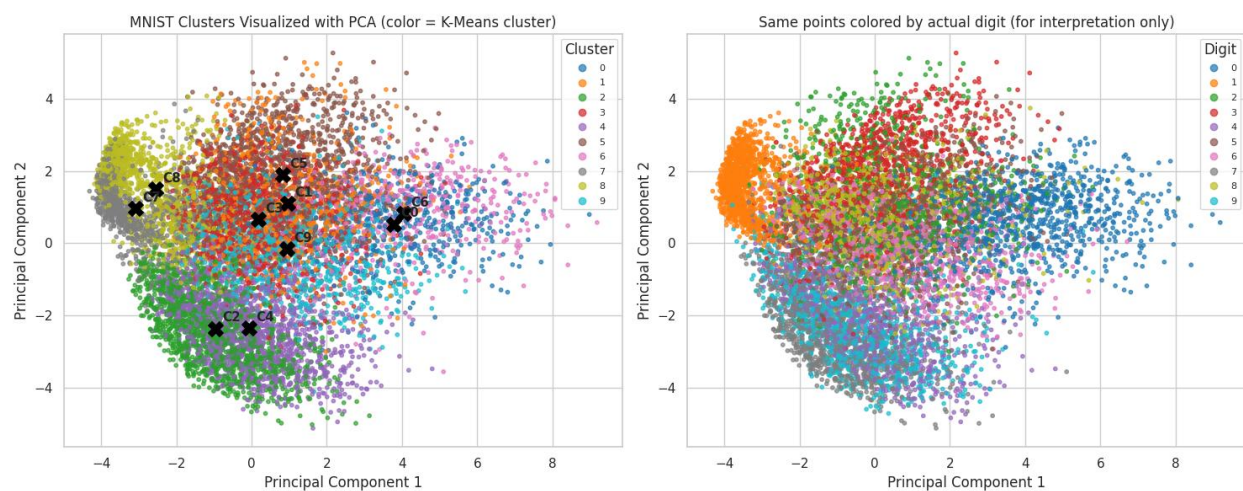


Figure 16. PCA (PC1 vs PC2) colored by K-Means cluster (left, black X = centroids) and by actual digit (right).

1. Do the clusters appear separated?

Only partly. The two components keep just **16.9%** of the variance (PC1 9.8%, PC2 7.1%).

2. Are some clusters overlapping?

Yes, heavily in the center: C1, C3, C5 and C9 share one area, and C2 and C4 overlap at the bottom.

3. Which groups appear to be more clearly separated?

The 1-clusters (C7, C8) on the far left and the 0-clusters (C0, C6) on the right. PC1 roughly separates thin, low-ink shapes from round, high-ink shapes. The 4/7/9 clusters form their own band at the bottom.

4. Does the PCA visualization support what you observed from the cluster images?

Yes. The two 0-clusters sit almost on top of each other, as do the two 1-clusters, confirming the style split. C2 and C4 are neighbors (4/7/9 mix), and C3 and C5 overlap (3/5/8 mix). Because 2D PCA loses 83% of the variance, the t-SNE view (Section 5.5) shows the digit structure much more clearly.

6.6 Part 6: Predict the Cluster of New Images

Ten images that were not in the 10,000-image training sample (indices 10000–10009) were assigned with `kmeans.predict()`.

Image	10000	10001	10002	10003	10004	10005	10006	10007	10008	10009
Predicted cluster	5	3	2	4	4	0	7	8	5	1
Actual digit	3	8	7	9	9	0	1	1	5	2

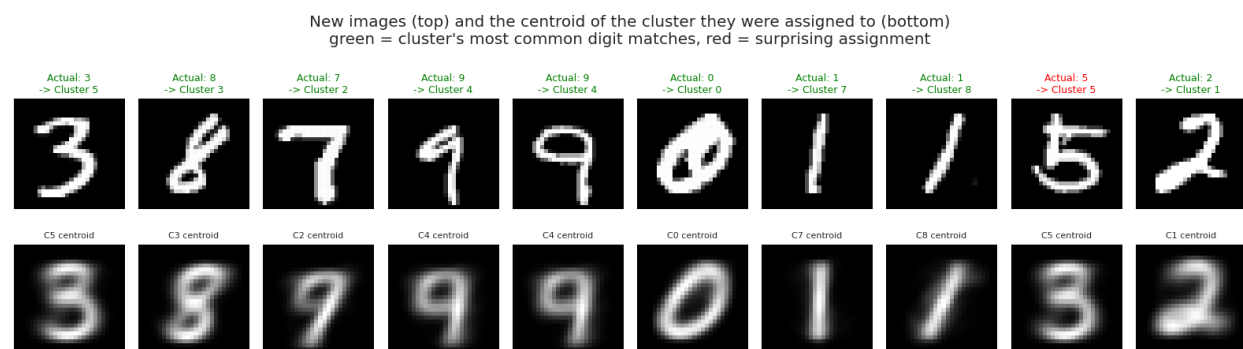


Figure 17. New images (top) and the centroid of their assigned cluster (bottom).

1. Which cluster was assigned to each image?

See the table above. In **9 of 10** cases the image landed in a cluster whose most common digit matches it.

2. Do images that look similar receive similar cluster assignments?

Yes. Both 9s went to C4. The upright 1 went to C7 and the slanted 1 to C8, so the model groups by appearance, including slant.

3. Are any assignments surprising?

Image 10008, a 5, went to C5, which mostly holds 3s (the matching number is a coincidence). Its curved lower half looks like a 3, and its distance to the centroid (6.98) was among the largest, so it fit poorly.

4. Why might K-Means sometimes assign an image to an unexpected cluster?

It always picks the nearest blurry average by pixel distance. An unusual style can be closer to another digit's average, some clusters are already mixed, and there is no "none of these" option.

6.7 Part 7 : Final Analysis

1. What did K-Means discover in the MNIST dataset?

Real but partial structure: groups of images with similar shape and writing style. Labeling each cluster with its most common digit matches the true digit **59.1%** of the time on the sample (ARI 0.362, NMI 0.496), compared with 10% for random guessing. The 60,000-image model gives almost the same result (59.5% on unseen test images).

2. What value of K did you choose, and why?

K = 10: the inertia gain per extra cluster roughly halves after 10, gains level off near 10 in the $k = 2-15$ search, and it matches the 10 digits.

3. What did the cluster centroids tell you?

Sharp centroids marked pure clusters, while blurry blends (7 + 9, 9 + 4, 8 + 3 + 5) marked mixed ones. They also showed that K-Means splits digits by style (round vs slanted 0s, upright vs slanted 1s).

4. Did the clusters correspond closely to particular digits?

Partly. Five clusters are 71%–89% one digit, and five are mixed. 4 and 5 had no cluster of their own in the sample model (5 and 9 in the 60,000-image model).

5. Which digits were difficult to separate?

The families {4, 7, 9} and {3, 5, 8}; 5 was the hardest. 0, 1, 2 and 6 were the easiest.

6. What did the PCA visualization show?

Heavy overlap in 2D (about 17% of the variance kept), with 0s and 1s at opposite ends and the 4/7/9 band apart, which matches the centroids and crosstab. The optional t-SNE view separated the digits much more clearly (trustworthiness 0.982 vs 0.739).

7. What are some limitations of using K-Means for handwritten digit images?

K must be chosen in advance; raw pixel distance is sensitive to shift, slant and thickness; it assumes round, similar-sized clusters; every image is forced into a cluster; cluster IDs need human interpretation; and results depend on the random start (controlled here with `n_init = 10` and `random_state = 42`).

8. How is this project different from Project 3, where you used supervised learning?

Project 3 trained on a known target (Buy, Sell or Hold) and was judged on how often it predicted that answer. Project 4 had no target: K-Means grouped images using pixels alone, and the labels were only used afterward to interpret the groups. Supervised learning answers "what is this?", while unsupervised learning answers "what belongs together?".

7. Insights

1. **K-Means finds real but partial structure.** About 59% accuracy without labels is far above random, but it does not replace supervised learning.
2. **It groups by appearance, not meaning.** It spends two clusters on slanted vs upright 0s and 1s while look-alike digits share a cluster.
3. **Two families cause most errors: {4, 7, 9} and {3, 5, 8}.** The confusion matrix, centroid distances, dendrogram and boundary analysis all agree.
4. **The model generalizes within MNIST.** Train and test behave almost identically.
5. **"Good fit" is not "correct".** The external data was 100% good fit but only 33.6% accurate. Distance checks catch noise but miss style shifts.
6. **t-SNE is the better view.** It keeps 98% of local neighborhoods; 2D PCA keeps only 17% of the variance and can mislead.
7. **The distance measure is the bottleneck.** Clustering in t-SNE space reaches 81.9% accuracy, so the digits are separable, just not by pixel distance.
8. **The confidence margin is useful.** Low-margin images sit on cluster borders and are the best candidates for human review.

8. Limitations

- k was chosen partly with domain knowledge; the metrics alone did not point clearly to 10.
- Majority mapping needs labels and hides digits that are never a majority (5 and 9).
- Samples were used for the k search, silhouette and t-SNE, so results could shift slightly with another sample.
- t-SNE cannot transform new images, and distances between its islands are not meaningful.
- K-Means assumes round, similar-sized clusters, which handwritten digits do not have.
- The fit threshold is distance-based and does not detect style or domain shift.

9. Recommendations and Next Steps

- **Use more clusters (k = 15-20)** and map several clusters to each digit so 5 and 9 can appear.
- **Cluster in a non-linear embedding that handles new data**, such as UMAP or an autoencoder.
- **Compare other algorithms:** Gaussian Mixture Models, DBSCAN/HDBSCAN and Agglomerative clustering.
- **Use the margin for human-in-the-loop labeling**, sending the least confident images for review.
- **Monitor new data sources with a small labeled sample**, since distance alone missed the external-data drop.

10. Conclusion

Can unsupervised learning discover groups in handwritten digits? Yes, partly. K-Means on normalized MNIST pixels found ten meaningful groups that match the real digits about 59% of the time, behave the same on unseen images, and stay stable under noise. The analysis shows why it falls short: the method groups by pixel similarity, so it splits digits by writing style and merges look-alike digits (4/7/9 and 3/5/8). Dimensionality reduction confirmed this: t-SNE revealed clear digit structure (trustworthiness 0.982), and clustering in that space raised accuracy to 81.9%. The key lesson is that **how the data is represented matters more than the clustering algorithm itself**.

Appendix: Saved Outputs and Reproducibility

- **results/**: 21 CSV tables covering EDA, the elbow search, cluster profiles, fit thresholds, validation, the confusion matrix, per-image predictions, centroids and 2D coordinates for Power BI or Tableau.
- **models/kmeans_k10.joblib** (0.27 MB) and **models/model_info.json** (cluster-to-digit mapping, fit thresholds, preprocessing). A reload test confirmed identical predictions.
- All files are under GitHub's 100 MB limit.
- The notebook follows Parts 1–7 of the updated instructions, with a Questions and Answers cell after each part. The extended analysis (60,000-image model, validation and t-SNE) follows Part 6, and Part 7 (Final Analysis) comes before the results are saved.
- To reproduce: open the notebook and run **Kernel → Restart & Run All**. All random steps use seed 42.